Informe técnico de la práctica con Apache Hive

Laboratorio LI16

Estudiante: Ronald Joel Saavedra Vargas

Materia: Tecnologías Emergentes I

Fecha de ejecución: 20 de octubre de 2026

Resultado del laboratorio

Se implementó un entorno propio de cuatro contenedores, se cargaron 99.003 usuarios en db_ronald.fb_ronald y se resolvieron las siete consultas requeridas. Los resultados de Hive coinciden con una verificación independiente del CSV original.

El desarrollo mantiene la numeración de la guía y destaca cada requerimiento en amarillo. Las evidencias distinguen la interfaz real de HDFS del visor de registros de comandos y SQL. En este último, la fecha de ejecución y la hora de captura se muestran por separado.

Objetivo

Implementar Apache Hive sobre HDFS mediante contenedores Docker, cargar el dataset pseudo Facebook y responder siete preguntas mediante HiveQL. El desarrollo sigue los puntos de la guía LI16 y registra los comandos, resultados y análisis de cada etapa.

Entorno y decisiones

La práctica se ejecutó con Windows como anfitrión, Ubuntu 24.04.5 LTS en WSL 2 y Docker Desktop con motor Linux. Se creó el repositorio Git local /root/li16-ronald en Ubuntu con una configuración propia de Docker Compose. No se clonó el repositorio Big Data Europe. La copia del código para revisión está en la carpeta proyecto del trabajo.

Arquitectura

Se utilizaron las imágenes oficiales apache/hadoop:3.4.1 y apache/hive:4.0.1. HDFS dispone de un NameNode y un DataNode; Hive dispone de un metastore persistente en Derby y un HiveServer2. Las consultas se ejecutan con MapReduce local, sin un clúster YARN. Este despliegue académico usa replicación de uno y permisos HDFS simplificados dentro de la red local de contenedores.

Adaptaciones de la guía

Docker Compose se invoca como docker compose. Los contenedores se llaman li16-ronald-namenode-1, li16-ronald-datanode-1, li16-ronald-hive-metastore-1 y li16-ronald-hive-server-1. El cliente SQL es Beeline con JDBC a HiveServer2. La interfaz del NameNode usa el puerto 9870 de Hadoop 3. Estas diferencias se deben a la arquitectura propia y conservan el flujo CSV, contenedor, HDFS y tabla Hive.

Trazabilidad de las evidencias

Los registros completos se conservan en registros como archivos JSON con comando, salida, fecha y código de salida. El visor local presenta esos resultados y conserva las advertencias originales en una sección desplegable. Las capturas de HDFS incluyen un reloj añadido a la interfaz local que lee la hora real del sistema del navegador en UTC-4; este reloj no modifica los datos de HDFS.

Punto 1 Revisar los contenedores existentes

Comando de verificación

```
Get-Date; docker ps -a
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/01.json.

Interpretación

Se ejecutó docker ps -a antes de iniciar el laboratorio. Se encontraron greenbit-frontend, greenbit-backend y greenbit-db, todos detenidos desde un proyecto anterior. La ausencia de contenedores en docker ps significa que no había servicios activos, no que Docker estuviera vacío. Se conservaron los contenedores existentes y se aisló el laboratorio bajo el proyecto li16-ronald.

Punto 2 Crear un repositorio diferente de Big Data Europe

Comando de verificación

```
wsl -d Ubuntu-24.04 -u root -- sh -lc 'cd /root/li16-ronald; pwd; git log -2 --oneline; git ls-files; docker compose config --services'
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/02.json.

Interpretación

Se creó un repositorio Git propio en /root/li16-ronald dentro del sistema de archivos Linux. El repositorio contiene compose.yaml, la configuración XML de Hadoop y Hive y las consultas SQL. No tiene un remoto publicado en GitHub. La creación local sustituye la clonación del ejemplo porque se eligió desarrollar una configuración propia, distinta a Big Data Europe.

Punto 3 Levantar los servicios con Docker Compose

Comando de verificación

```
wsl -d Ubuntu-24.04 -u root -- sh -lc 'cd /root/li16-ronald; docker compose up -d; docker compose ps'
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/03.json.

Interpretación

Desde el directorio del proyecto se ejecutó docker compose up -d. Compose creó una red propia y volúmenes para los metadatos del NameNode, los bloques del DataNode y el metastore. Durante la preparación se corrigió la ruta persistente de Derby y se añadió una comprobación de salud del metastore para que HiveServer2 espere a que el servicio esté disponible.

Punto 4 Listar los contenedores del laboratorio

Comando de verificación

```
docker ps --filter label=com.docker.compose.project=li16-ronald
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/04.json.

Interpretación

La consulta docker ps filtrada por el proyecto li16-ronald muestra los cuatro contenedores activos. El NameNode publica 127.0.0.1:9870, mientras HiveServer2 publica 127.0.0.1:10000 y 127.0.0.1:10002. El DataNode y el metastore se comunican por la red interna de Compose y no necesitan publicar puertos al anfitrión.

Punto 5 Analizar la función de cada contenedor

NameNode

El NameNode administra el espacio de nombres de HDFS: directorios, archivos, permisos y ubicación de bloques. En esta práctica permite listar warehouse y expone la interfaz HTTP del puerto 9870. Se relaciona con el concepto de metadatos del sistema distribuido; no es el servicio que almacena los bytes del CSV como bloques.

DataNode

El DataNode conserva los bloques del CSV y atiende las lecturas y escrituras solicitadas a HDFS. Informa su estado y sus bloques al NameNode. El reporte muestra un DataNode activo y replicación de uno, suficiente para demostrar el flujo académico, aunque sin tolerancia a la pérdida de ese único nodo.

Hive Metastore

El metastore registra las bases, tablas, columnas y ubicaciones de los datos. HiveServer2 consulta este catálogo para convertir una sentencia SQL en operaciones sobre archivos HDFS. En este despliegue mantiene sus metadatos en Derby dentro de un volumen persistente y expone internamente el servicio Thrift en el puerto 9083.

HiveServer2

HiveServer2 recibe las consultas JDBC de Beeline, utiliza el catálogo del metastore y lee los archivos de HDFS. Permite que el usuario trabaje con tablas y agregaciones sin programar directamente cada lectura de bloques. En el laboratorio se usa ejecución MapReduce local para procesar un dataset pequeño sin desplegar YARN.

Comando de verificación

```
docker ps --filter label=com.docker.compose.project=li16-ronald --format 'table {{.Names}}\
t{{.Image}}\t{{.Status}}'; docker exec li16-ronald-namenode-1 hdfs dfsadmin -report
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/05.json.

Punto 6 Acceder al shell del NameNode

Comando de verificación

```
docker exec li16-ronald-namenode-1 /bin/bash -c 'hostname; whoami; pwd; hdfs version | head -1'
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/06.json.

Interpretación

Se ejecutó /bin/bash dentro de li16-ronald-namenode-1 con docker exec. La salida identifica el hostname namenode, el usuario root y el directorio /opt/hadoop. Esto permite distinguir el sistema anfitrión del contenedor: el comando hdfs que sigue se ejecuta dentro del servicio que dispone de la configuración del sistema de archivos distribuido.

Punto 7 Listar el warehouse inicial de HDFS

Comando de verificación

```
docker exec li16-ronald-namenode-1 hdfs dfs -ls /user/hive/warehouse/
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/07.json.

Interpretación

La consulta hdfs dfs -ls /user/hive/warehouse/ terminó sin errores y sin entradas. El directorio se había creado para preparar el laboratorio y todavía no contenía el CSV ni las carpetas de tablas. Una salida vacía en esta etapa es coherente con el estado inicial; posteriormente el directorio cambia al subir el archivo y crear la base de datos.

Punto 8 Validar dirección y puerto del NameNode

Comando de verificación

```
docker port li16-ronald-namenode-1; (Invoke-WebRequest http://localhost:9870 -
UseBasicParsing).StatusCode
```

Evidencia

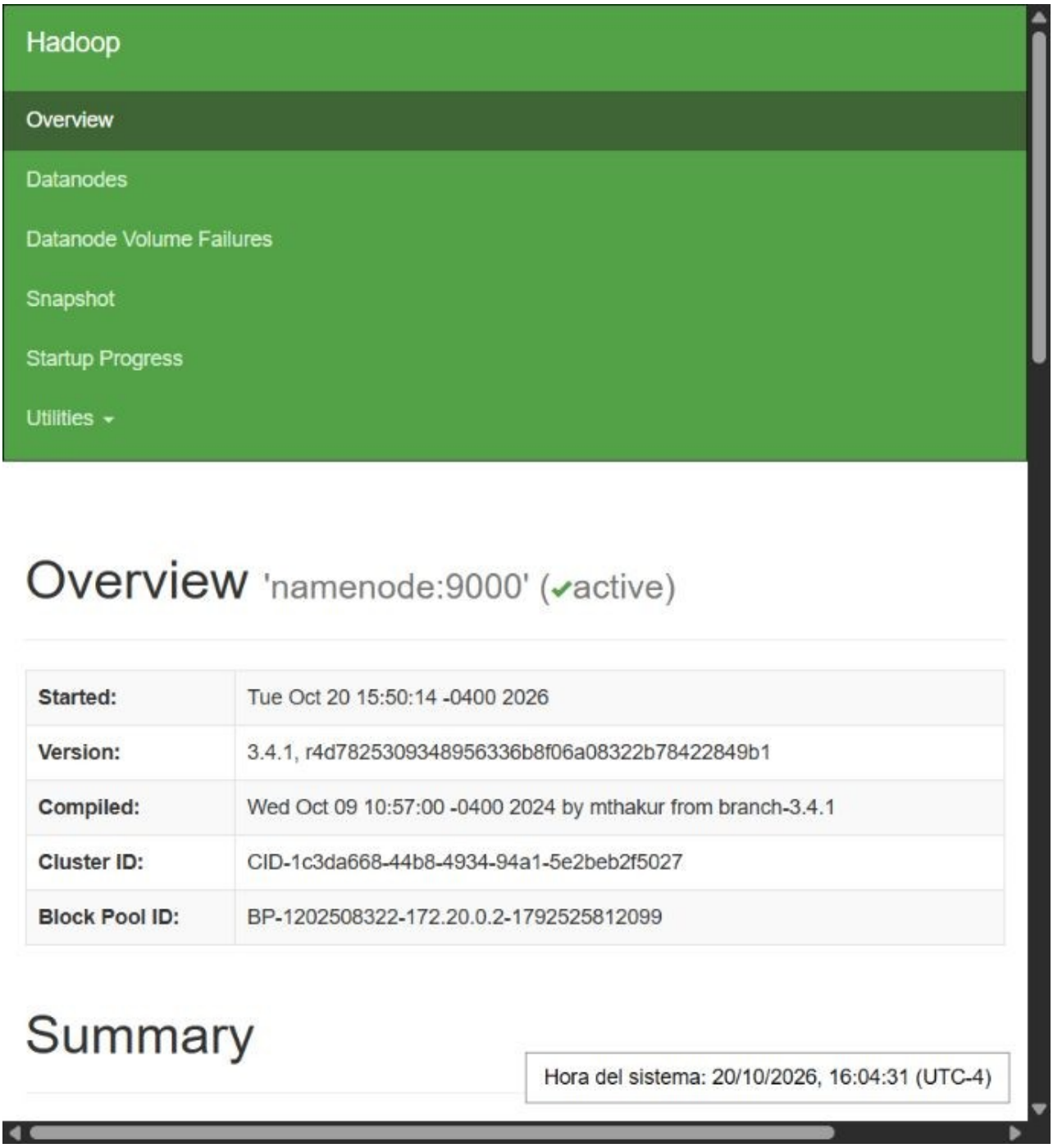


Figura 08-web Interfaz web real de HDFS

Interpretación

docker port muestra 9870/tcp -> 127.0.0.1:9870. El primer 9870 es el puerto del servicio en el contenedor y el último es el puerto publicado en Windows. Desde este mismo equipo la dirección del anfitrión es 127.0.0.1 y la interfaz se encuentra en http://localhost:9870/dfshealth.html. Se confirmó una respuesta HTTP 200. La publicación se limita a la interfaz de loopback del equipo.

Punto 9 Revisar el contenido de warehouse por la interfaz web

Comando de verificación

```
(Invoke-WebRequest 'http://localhost:9870/webhdfs/v1/user/hive/warehouse/?op=LISTSTATUS' -
UseBasicParsing).Content
```

Evidencia

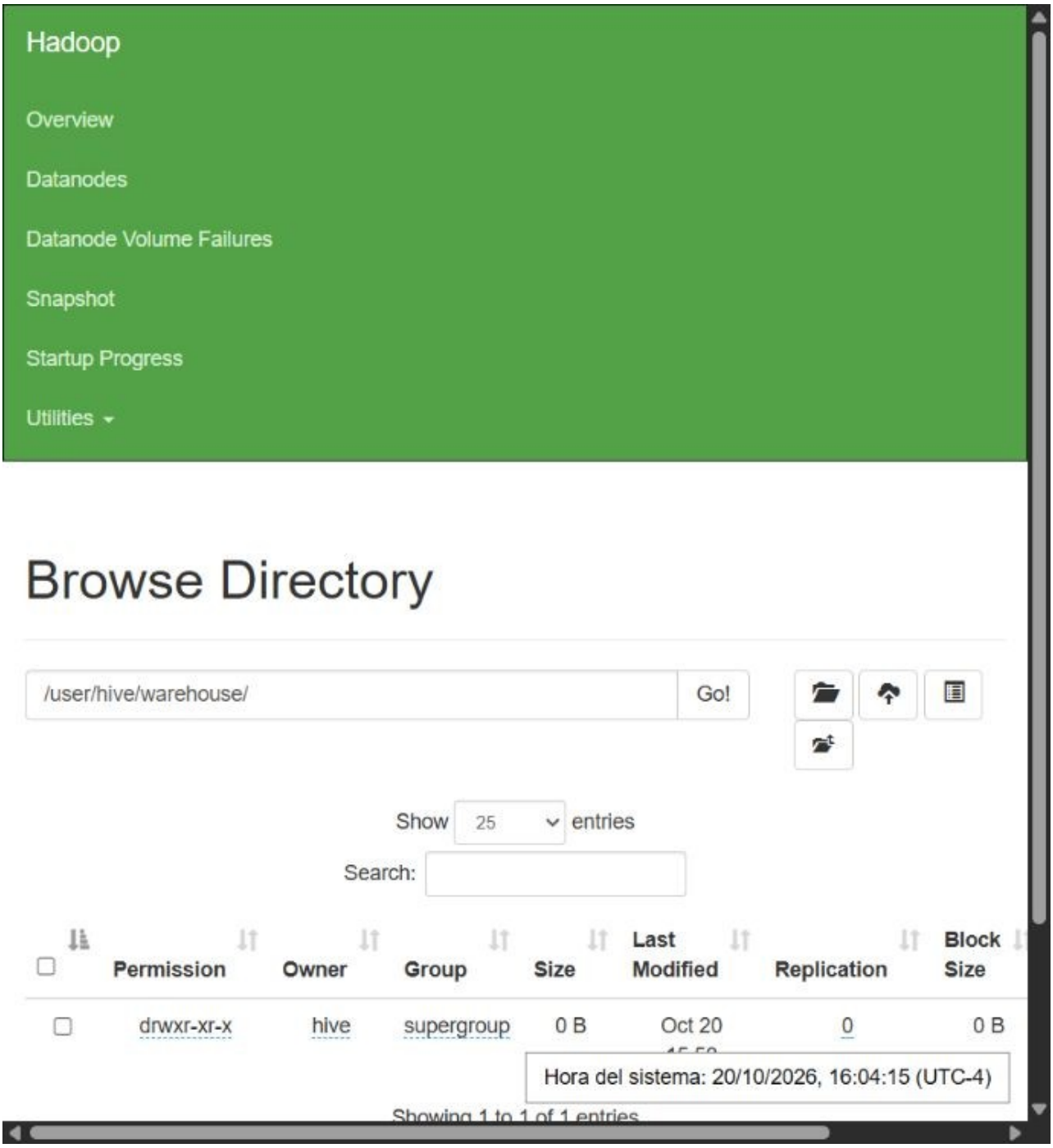


Figura 09-web Interfaz web real de HDFS

Interpretación

La ruta de exploración es http://localhost:9870/explorer.html#/user/hive/warehouse/. La consulta de la API WebHDFS LISTSTATUS confirmó el directorio db_ronald.db después de crear la tabla. El CSV inicialmente subido dejó de estar suelto en warehouse porque LOAD DATA lo trasladó al directorio de la tabla. La respuesta de WebHDFS es una verificación del servicio HTTP; no sustituye una captura de navegación de Utilities y Browse the file system.

Punto 10 Descargar y renombrar el dataset

Comando de verificación

```
Get-Date; Get-Item datos/facebook-data.zip,datos/original/pseudo_facebook.csv; Get-FileHash
datos/original/pseudo_facebook.csv -Algorithm SHA256
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/10.json.

Interpretación

Se descargó el archivo ZIP desde la API pública de Kaggle del dataset Facebook Data de Sheena Batra, asociado al dataset pseudo_facebook.csv utilizado por la guía. El ZIP se guardó en datos/facebook-data.zip y el original se conservó en datos/original/pseudo_facebook.csv. La copia preparada lleva el nombre pseudo_facebook_ronald.csv. Se registró el hash SHA256 para identificar el archivo exacto utilizado.

Punto 11 Quitar el encabezado del CSV

Comando de verificación

```
& 'C:/Users/wiget/.cache/codex-runtimes/codex-primary-runtime/dependencies/python/python.exe'
scripts/preparar_datos.py
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/11.json.

Interpretación

Se leyó el archivo con un lector CSV, se separó el encabezado y se escribió una copia sin la primera fila. Se comprobaron 15 campos en cada uno de los 99.003 registros y 99.003 identificadores distintos. El original no se modificó. Esta preparación evita que los nombres de columna se interpreten como datos al cargar la tabla de texto delimitado.

Punto 12 Copiar el archivo al contenedor NameNode

Comando de verificación

```
docker cp datos/pseudo_facebook_ronald.csv li16-ronald-namenode-1:/tmp/; docker exec li16-ronald-namenode-1 ls -lh /tmp/pseudo_facebook_ronald.csv
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/12.json.

Interpretación

docker cp copió datos/pseudo_facebook_ronald.csv al directorio /tmp del NameNode. La verificación con ls -lh confirmó el archivo de aproximadamente 5 MB. Esta copia todavía pertenece al sistema de archivos del contenedor; no forma parte de HDFS hasta ejecutar el comando de carga del siguiente punto.

Punto 13 Subir el CSV a HDFS y verificarlo

Comando de verificación

```
docker exec li16-ronald-namenode-1 /bin/bash -c 'cd /tmp; hdfs dfs -put pseudo_facebook_ronald.csv
/user/hive/warehouse/; hdfs dfs -ls /user/hive/warehouse/'
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/13.json.

Interpretación

Dentro del NameNode se ejecutó hdfs dfs -put desde /tmp hacia /user/hive/warehouse/. El listado confirmó un archivo de 5.216.842 bytes, propietario root y replicación uno. Ahora los bytes se almacenan como bloques en el DataNode y el NameNode administra su nombre y ubicación. El reporte de HDFS confirmó un DataNode vivo y ningún bloque faltante.

Punto 14 Acceder al shell del servidor Hive

Comando de verificación

```
docker exec li16-ronald-hive-server-1 /bin/bash -c 'hostname; whoami; pwd; beeline --version'
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/14.json.

Interpretación

Se ejecutó /bin/bash en li16-ronald-hive-server-1. El hostname hive-server y el directorio /opt/hive confirman que se está en otro contenedor. Beeline identificó Hive 4.0.1. El acceso SQL se realiza a jdbc:hive2://localhost:10000 desde ese contenedor, donde localhost se refiere al propio servidor Hive.

Punto 15a Crear la base de datos personalizada

Consulta ejecutada

```
CREATE DATABASE IF NOT EXISTS db_ronald; SHOW DATABASES;
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/15a.json.

Interpretación

CREATE DATABASE IF NOT EXISTS db_ronald creó el espacio de nombres del laboratorio. SHOW DATABASES mostró db_ronald junto con default. El nombre personalizado permite separar la tabla de otros objetos que puedan existir en Hive y cumple la identificación solicitada por la guía.

Punto 15b Identificar columnas y tipos de datos

Columna	Tipo Hive
userid	BIGINT
age	INT
dob_day	INT
dob_year	INT
dob_month	INT
gender	STRING
tenure	INT
friend_count	INT
friendships_initiated	INT
likes	BIGINT
likes_received	BIGINT
mobile_likes	BIGINT
mobile_likes_received	BIGINT
www_likes	BIGINT
www_likes_received	BIGINT

Comando de verificación

```
Get-Content datos/original/pseudo_facebook.csv -TotalCount 2; Get-Content proyecto/sql/15c.sql
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/15b.json.

Interpretación

El CSV tiene 15 columnas y el esquema conserva exactamente su orden. userid usa BIGINT; las edades, fechas, antigüedad y cantidades de amistades usan INT; gender usa STRING y los contadores de likes usan BIGINT. Se configura NA como marcador nulo. La validación posterior detectó 175 géneros ausentes y dos valores tenure ausentes, por lo que esos valores no deben interpretarse como ceros.

Punto 15c Crear la tabla fb_ronald

Consulta ejecutada

```
CREATE TABLE IF NOT EXISTS db_ronald.fb_ronald (
userid BIGINT, age INT, dob_day INT, dob_year INT, dob_month INT,
gender STRING, tenure INT, friend_count INT, friendships_initiated INT,
likes BIGINT, likes_received BIGINT, mobile_likes BIGINT,
mobile_likes_received BIGINT, www_likes BIGINT, www_likes_received BIGINT
) ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE TBLPROPERTIES ('serialization.null.format'='NA');
DESCRIBE db_ronald.fb_ronald;
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/15c.json.

Interpretación

Se creó db_ronald.fb_ronald con ROW FORMAT DELIMITED, separador coma y almacenamiento TEXTFILE. DESCRIBE confirmó las 15 columnas. Los nombres originales friend_count y likes_received facilitan seguir la semántica del dataset. Hive 4 registró la tabla no transaccional como EXTERNAL_TABLE mediante traducción automática, condición que se verificó con DESCRIBE FORMATTED.

Punto 15d Verificar la ubicación de la tabla

Consulta ejecutada

```
DESCRIBE FORMATTED db_ronald.fb_ronald;
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/15d.json.

Interpretación

DESCRIBE FORMATTED devolvió Location hdfs://namenode:9000/user/hive/warehouse/db_ronald.db/fb_ronald. La ubicación llega hasta el directorio de la tabla, como pide la guía. Esta ruta distingue la base de datos db_ronald.db del directorio fb_ronald y permite comprobar dónde deben quedar los archivos después de la carga.

Punto 15e Cargar el archivo en la tabla

Consulta ejecutada

```
LOAD DATA INPATH '/user/hive/warehouse/pseudo_facebook_ronald.csv' INTO TABLE db_ronald.fb_ronald;
```

Evidencia

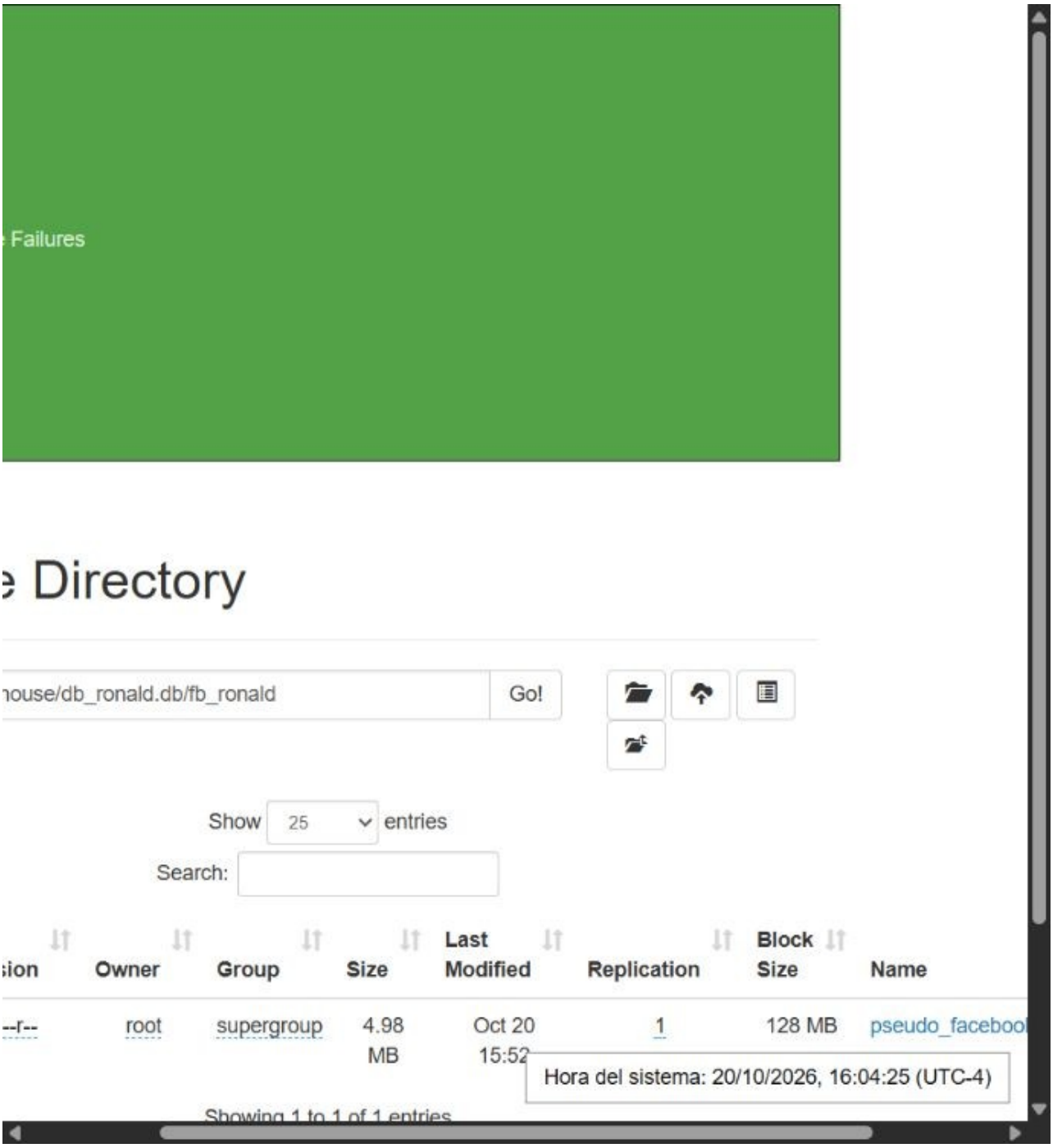


Figura 15e-web Interfaz web real de HDFS

Interpretación

LOAD DATA INPATH tomó el CSV desde /user/hive/warehouse/pseudo_facebook_ronald.csv y lo trasladó al directorio de db_ronald.fb_ronald. La sentencia terminó con código cero; el listado posterior verificó el archivo en db_ronald.db/fb_ronald. La lectura de diez filas y el conteo total confirmaron que la carga produjo datos consultables.

Punto 15f Mostrar diez registros

Consulta ejecutada

```
SELECT * FROM db_ronald.fb_ronald LIMIT 10;
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/15f.json.

Interpretación

SELECT * FROM db_ronald.fb_ronald LIMIT 10 devolvió diez filas con los quince campos. Se observan identificadores numéricos, edades y valores de género coherentes con el CSV. LIMIT limita la cantidad de filas, pero no establece un orden; por ello se trata de una muestra de diez registros y no de una clasificación del dataset.

Punto 16a Determinar el total de usuarios

Consulta ejecutada

```
SELECT COUNT(*) AS total_usuarios, COUNT(DISTINCT userid) AS usuarios_distintos FROM
db_ronald.fb_ronald;
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/16a.json.

Interpretación

Hive devolvió 99.003 filas y 99.003 identificadores de usuario distintos. La coincidencia entre COUNT y COUNT DISTINCT confirma que cada fila representa un usuario diferente en este archivo. El resultado también coincide con el conteo independiente del CSV preparado, de modo que no se perdió ni duplicó ningún registro durante la carga.

Punto 16b Determinar usuarios mayores de 25 años

Consulta ejecutada

```
SELECT COUNT(*) AS usuarios_mayores_25 FROM db_ronald.fb_ronald WHERE age > 25;
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/16b.json.

Interpretación

La condición age > 25 devolvió 56.676 usuarios, aproximadamente el 57,25 % de la muestra. Se excluyeron las personas de exactamente 25 años porque el enunciado indica por encima de 25. El grupo complementario de 13 a 25 años contiene 42.327 registros.

Punto 16c Comparar el número de amigos por género

Consulta ejecutada

```
SELECT gender, COUNT(*) AS usuarios, ROUND(AVG(friend_count),2) AS promedio_amigos FROM
db_ronald.fb_ronald WHERE gender IN ('male','female') GROUP BY gender ORDER BY promedio_amigos DESC;
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/16c.json.

Interpretación

Las 40.254 usuarias con género female tienen un promedio de 241,97 amigos, mientras los 58.574 usuarios male promedian 165,04. En esta muestra el promedio femenino es mayor por 76,93 amigos. La comparación usa promedios para que el distinto tamaño de los grupos no determine el resultado; los 175 registros sin género se excluyen de esta comparación.

Punto 16d Comparar likes recibidos por grupos de edad

Consulta ejecutada

```
SELECT CASE WHEN age BETWEEN 13 AND 25 THEN '13 a 25' ELSE 'mayores de 35' END AS grupo, COUNT(*) AS usuarios, SUM(likes_received) AS likes_recibidos, ROUND(AVG(likes_received),2) AS promedio_recibidos FROM db_ronald.fb_ronald WHERE age BETWEEN 13 AND 25 OR age > 35 GROUP BY CASE WHEN age BETWEEN 13 AND 25 THEN '13 a 25' ELSE 'mayores de 35' END ORDER BY grupo;
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/16d.json.

Interpretación

Los 42.327 jóvenes de 13 a 25 años recibieron 8.477.550 likes, con un promedio de 200,29 por usuario. Los 38.037 mayores de 35 recibieron 3.982.359, con un promedio de 104,70. Tanto el total como el promedio son mayores entre los jóvenes. Los usuarios de 26 a 35 años quedan fuera de esta comparación y no se extrapola el resultado a toda la población de Facebook.

Punto 16e Contar usuarios por mes de cumpleaños

Consulta ejecutada

```
SELECT dob_month AS mes_cumpleanios, COUNT(*) AS usuarios FROM db_ronald.fb_ronald GROUP BY dob_month
ORDER BY dob_month;
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/16e.json.

Interpretación

GROUP BY dob_month produjo doce grupos cuya suma es 99.003. Enero registra la mayor cantidad, 11.772 usuarios, y noviembre la menor, 7.205. El resultado cuenta el mes de nacimiento declarado y no actividad mensual en Facebook. Una distribución desigual de cumpleaños por sí sola no explica sus causas.

Mes	Usuarios
1	11772
2	7632
3	8110
4	7810
5	8271
6	7607
7	8021
8	8266
9	7939
10	8476
11	7205
12	7894

Punto 16f Comparar celular y computadora entre jóvenes

Consulta ejecutada

```
SELECT COUNT(*) AS usuarios_13_25, SUM(mobile_likes) AS likes_celular, SUM(www_likes) AS likes_web,
ROUND(AVG(mobile_likes),2) AS promedio_celular, ROUND(AVG(www_likes),2) AS promedio_web FROM
db_ronald.fb_ronald WHERE age BETWEEN 13 AND 25;
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/16f.json.

Interpretación

Entre los 42.327 jóvenes se registraron 5.248.117 likes dados desde celular y 2.349.153 desde la web, con promedios de 123,99 y 55,50 respectivamente. La actividad de dar likes es mayor desde celular. El dataset no mide tiempo conectado ni el dispositivo físico de toda sesión web; por eso esta conclusión describe likes por plataforma y usa web como aproximación a computadora, sin afirmar tiempo total de uso.

Punto 16g Comparar celular y computadora entre adultos

Consulta ejecutada

```
SELECT COUNT(*) AS usuarios_mayores_25, SUM(mobile_likes) AS likes_celular, SUM(www_likes) AS
likes_web, ROUND(AVG(mobile_likes),2) AS promedio_celular, ROUND(AVG(www_likes),2) AS promedio_web
FROM db_ronald.fb_ronald WHERE age > 25;
```

Evidencia

Captura pendiente de incorporar. El registro de ejecución está disponible en registros/16g.json.

Interpretación

Entre los 56.676 mayores de 25 se registraron 5.257.715 likes dados desde celular y 2.597.277 desde la web. Los promedios respectivos son 92,77 y 45,83 por usuario. También en este grupo predominan los likes desde celular. La comparación utiliza likes dados, no mobile_likes_received, porque los likes recibidos describen a quien interactuó con la persona y no su propia plataforma de actividad.

Validación de datos y límites del análisis

Se contrastaron las siete consultas de Hive con cálculos independientes sobre el CSV original. Coinciden los conteos, sumas y promedios. La tabla tiene 99.003 filas, cero identificadores nulos, 175 géneros nulos y dos valores tenure nulos. Las edades van de 13 a 113 años; este rango se describe como dato del archivo, sin verificar que cada edad represente una edad real.

Las consultas de género excluyen los valores desconocidos. Los límites de edad siguen los enunciados: jóvenes entre 13 y 25 inclusive, adultos mayores de 25 y el grupo de comparación de likes mayores de 35. Los resultados describen esta muestra histórica, no el comportamiento actual de todos los usuarios de Facebook.

Conclusiones

El laboratorio mostró la separación entre almacenar datos en HDFS, registrar su estructura en el metastore y consultarlos mediante HiveServer2. La ruta del CSV cambió al cargarlo en la tabla, mientras sus 99.003 registros se conservaron. En este dataset las mujeres tienen más amigos en promedio, los jóvenes reciben más likes en promedio que los mayores de 35 y los dos grupos analizados dan más likes desde celular que desde la web.

Archivos para reproducir la práctica

El directorio proyecto contiene la configuración Compose y las consultas SQL. datos conserva el ZIP de Kaggle, el CSV original y la copia sin encabezado. registros contiene las salidas completas y evidencias las capturas. scripts contiene la preparación del dataset, la verificación independiente y la generación de este informe. El repositorio de ejecución está en /root/li16-ronald de Ubuntu.

Referencias

Universidad del Valle. LI16 Práctica con Apache Hive 1. Guía proporcionada para el laboratorio.

Apache Hive. Setting Up Hive with Docker. https://hive.apache.org/docs/latest/admin/setting-up-hive-with-docker/

Apache Hadoop 3.4.1. HDFS Commands Guide. https://hadoop.apache.org/docs/r3.4.1/hadoop-project-dist/hadoop-hdfs/HDFSCommands.html

Sheena Batra. Facebook Data. Kaggle. https://www.kaggle.com/datasets/sheenabatra/facebook-data

Guía del dataset de la práctica. https://www.kaggle.com/code/youssefabdelghfar/facebook-data-analysis/input